Programació SCRIPTING Linux

Programació SCRIPTING Linux

La Shell de Linux és un intèrpret de comandes que permet a un usuari executar diferents tasques:

- **És un autèntic llenguatge de programació que permet a un administrador automatitzar i programar tasques.**
- **Com qualsevol llenguatge incorpora controls, condicions, bucles, variables i funcions.**
- **Un programa Shell o Bash no necessita ser compilat. És un llenguatge interpretat que s'executa en temps d'execució.**
- **Es pot crear i editar amb qualsevol editor (nano, vi …)**

Programació SCRIPTING Linux

Scripts bàsics

```
#!/bin/bash
echo "Bon dia "
```

```
#!/bin/bash
ls -la
```

Programació SCRIPTING Linux

Redireccionament a fitxer

```
#!/bin/bash
ls -l > redirect.txt
```

Redireccionament a null

```
#!/bin/bash
ls wkh*  2> /dev/null
```

Redireccionament d'errors a fitxer

```
#!/bin/bash
ls wkh*  2> errors.txt
```

Programació SCRIPTING Linux

Variables

Com en qualsevol llenguatge de programació, les variables s'utilitzen per poder guardar informació i a partir d'ella poder prendre decisions o realitzar operacions.

```
#!/bin/bash
numero=5
echo "el valor de la variable es "$numero
```

Programació SCRIPTING Linux

Operacions Aritmètiques i lògiques

Com en qualsevol llenguatge de programació es poden realitzar operacions aritmètiques i lògiques sobre les variables.

Per realitzar operacions s'utilitza la comanda **expr** i per realitzar comparacions s'utilitza també la comanda **test**

Programació SCRIPTING Linux

Operacions Aritmètiques i lògiques

```
#!/bin/bash
echo -n "Entra un valor: "
read var1
echo -n "Entra un altre valor: "
read var2
resultat=$(expr $var1 + $var2)
echo "El resultat de la suma és " $resultat
```

```
resultat=$(($var1+$var2))
resultat=$[$var1+$var2]
```

```
#!/bin/bash
echo -n "Entra un valor: "
read var1
echo -n "Entra un altre valor: "
read var2
resultat=$(expr $var1 = $var2)
echo "Resultat de la comparació: " $resultat
```

0 -> false 1 -> true

Programació SCRIPTING Linux

Estructures de control

Les estructures de control permeten canviar el fluxe d'un programa, en funció de l'estat de les variables.

Condició simple (IF)

```
#!/bin/bash
echo -n "Introduce un valor: "
read var
if  (( $var < 10 )); then
  echo "Es menor de 10"
else
  echo "Es major de 10"
fi
```

Programació SCRIPTING Linux

Condició simple (IF): exemple avaluar si exsiteix un fitxer

```
#!/bin/bash
FILE=/etc/passwd
  if [ -f $FILE ];
  then
     echo fitxer $FILE existeix
  else
     echo fitxer $FILE no trobat
  fi
```

```
#!/bin/bash
FILE=/etc/passwd
if  test -f $FILE
  then
     echo fitxer $FILE existeix
  else
     echo fitxer $FILE no trobat
  fi
```

Programació SCRIPTING Linux

Condicions múltiples (CASE)

```
#!/bin/bash
echo -n "Introduce un valor: "
read var1
case $var1 in
1) echo " uno ";;
2) echo " dos ";;
3) echo " tres ";;
4) echo " cuatro ";;
*) echo "opcion incorrecta ";;
esac
```

Programació SCRIPTING Linux

Bucle for

```
#!/bin/bash
for (( i = 0; i <= 5; i++ ))
  do
   echo " $i "
  done
```

Bucle for per recòrrer elements d'una llista

```
#!/bin/bash
for variable in lista_elementos
  do
    echo " $variable "
  done
```

Programació SCRIPTING Linux

Bucle while

El bucle while permet executar un codi fins que no es cumpleix una determinada condició de sortida

```
i=0;
while (test $limit -gt $i)
do
  echo “Valor” $i
  let i=$i+1
done
```

Funcions

Una funció és un bloc de codi que permet la seva reutilització d'una forma fàcil i senzilla.

```
#!/bin/bash
function mostrar() {
        echo “Bona tarda”
}

mostrar
```

```
#!/bin/bash
function suma (){
   resultat=$(expr $a + $b)
   echo "a + b =" $resultat
}
a=5
b=10
suma $a $b
```

(*) FUNCTION depen del sistema.

Programació SCRIPTING Linux

Funcions

Les funcions també poden retornar valors

```
#!/bin/bash
function suma (){
  c=$(expr $a + $b)
  return $c
}
a=5
b=10
suma $a $b
resultado=$?
echo $resultado
```

Programació SCRIPTING Linux

Funcions i variables locals

```
#!/bin/bash

HOLA=Hola

function hola() {
        local HOLA=Adeu
        echo $HOLA
}
echo $HOLA
hola
echo $HOLA
```